SE LAB 05 – ISSUES FIXED AND REFLECTION ANSWERS

NAME: PRAVEEN BALAJI P

SECTION: G

SRN: PES2UG24CS374

CLEANED AND UPDATED PYTHON PROGRAM

The updated helpdesk_system.py addresses the identified issues, including mutable default arguments, hard-coded credentials, unsafe shell execution, urgency validation, exception handling, file handling, input validation, and automatic execution during imports.

The final code contains these changes:

- Replacing the mutable audit-log default with None.
- Reading the administrator password from HELPDESK_ADMIN_PASSWORD.
- Validating ticket urgency to accept only integers from 1 to 5.
- Avoiding shell=True, validate hostnames, and set a timeout for subprocess execution.
- Validating user-provided fields.
- Using a context manager and UTF-8 encoding when saving JSON.
- Including an if __name__ == "__main__": guard.

ISSUES IDENTIFIED AND FIXES

No	Issue	Severity	Line(s)	Problem	Fix applied
1	Shell injection in ping	High (Bandit B602)	38	Hostname joined into a command run with shell=True, so input could run arbitrary commands	Removed shell=True; argument list; hostname regex validation; 10 s timeout; OS-specific ping flag
2	Hard-coded admin password	High (Bandit B105, rated Low by tool)	8	Password stored in plain text in the source	Read from HELPDESK_ADMIN_PASSWORD environment variable; closing disabled if unset
3	Mutable default argument	Medium (Pylint W0102)	10	Audit-log list default shared across all calls	Default None; log chosen inside the function
4	Bare except with pass	Medium (Pylint W0702, Bandit B110)	23	Every error in add_note silently swallowed	Removed; explicit checks return False
5	Urgency not validated	Medium (found by running)	-	Non-numeric or out-of-range urgency accepted	validate_urgency() allows integers 1 to 5 only

No	Issue	Severity	Line(s)	Problem	Fix applied
6	Missing input validation	Medium (review)	-	Empty or non-string user, issue, technician, note accepted	Added checks and trimming
7	Global counter and naming	Low (Pylint W0603, C0103)	7, 11	Counter changed through global and badly named	itertools.count with next()
8	File opened without with / encoding	Low (Pylint R1732, W1514)	41	File handle could leak; encoding platform-dependent	with open(..., encoding="utf-8")
9	== None and key iteration	Low (Pylint C0121, C0206)	32-33	Wrong None comparison; iterating keys then indexing	is None and .items()
10	Unused variable	Low (Pylint W0612)	47	t2 assigned but never used	Removed
11	Function not snake_case	Low (Pylint C0103)	30	unresolvedHighPriority	Renamed unresolved_high_priority
12	Missing docstrings	Low (Pylint C0114, C0116)	1, 10, 17, 20-45	Module and functions undocumented	Added docstrings
13	Long line and old string formatting	Low (Pylint C0301, C0209)	13, 14	118-character line; non-f-string formatting	Wrapped lines; used f-strings
14	Code ran on import	Low (review)	-	Demo logic executed whenever the module was imported	Moved into main() behind if __name__ == "__main__":

Severity labels are qualitative assessments of the original code issues, not necessarily the exact severity ratings assigned by the tools.

Final static analysis results

Tool	Before	After
Pylint	5.53 / 10 (21 messages)	**10.00 / 10**
Bandit	4 issues (1 High, 3 Low)	2 issues (0 High, 0 Medium, 2 Low)
Flake8	Baseline not captured	**0 violations**

Bandit still reports two Low findings, B404 (import subprocess) and B603 (subprocess.run). Both are generic warnings for any subprocess use; here the command is an argument list, no shell is used, the hostname is validated and a timeout is set. They were reviewed and documented rather than suppressed.

REFLECTION QUESTIONS AND ANSWERS

1. **Which fixes were easiest and hardest, and why?**

The easiest fixes were file-handling improvements and structural changes, such as using a context manager and adding a main-entry guard. These changes were relatively straightforward and improved code readability and maintainability.
The harder fixes involved replacing the hard-coded administrator password and improving subprocess execution. These required consideration of security, environment configuration, input validation, and safe command execution. Urgency validation also required handling invalid values consistently.

2. **Did any tools report false positives?**
   Bandit reported two Low-severity findings: B404 for importing subprocess and B603 for calling subprocess.run().
   These findings are warnings that require review rather than proof of an exploitable vulnerability. The application uses subprocess functionality for its ping feature, and the updated implementation includes safeguards such as avoiding shell=True, validating the hostname, and setting a timeout.
   The warnings were documented rather than suppressed. The final Bandit report contains no Medium- or High-severity findings.

3. **How could static analysis tools be integrated into the development workflow?**
   Pylint, Bandit, and Flake8 can be run before committing code and integrated into a continuous integration workflow using GitHub Actions. The tools can automatically analyze code whenever a developer pushes changes or creates a pull request.
   Developers can review the reports, fix genuine issues, and document any remaining warnings before merging changes. This helps detect problems early and improves consistency during code reviews.

4. **What tangible improvements were achieved?**
   The corrections improved the program's input validation, credential handling, subprocess safety, exception handling, file operations, and overall structure.
   The final Pylint score was 10.00/10, Flake8 reported no violations, and Bandit reported two Low-severity findings with no Medium- or High-severity findings. The remaining warnings were reviewed and documented, providing a transparent account of the final security analysis.